

URBAN FURNITURE

Accounting System

Frontend Tech Stack + Complete Page Requirements + Connected Flow

Purpose

A practical frontend blueprint based on the supplied Urban Furniture Accounting System PRD. It maps the required accounting workflow into pages, routes, dependencies and reusable UI patterns.

PRD direction	Frontend implementation
React + Tailwind CSS	Core UI framework and styling
English + Hindi + Gujarati	react-i18next localization
Master data drives transactions	Selectors and linked forms reuse Contacts, Products, Accounts and Journals
Double-entry accounting	Journal Entry UI shows debit/credit totals; backend remains source of truth
Reports	P&L, Balance Sheet and Budget Report consume posted accounting data

Recommended stack

React + TypeScript + Vite + Tailwind CSS + shadcn/ui + React Router + TanStack Query + React Hook Form + Zod + Recharts + react-i18next. React, Tailwind CSS and react-i18next are directly specified by the PRD; the other libraries are recommended implementation choices.

1. Recommended Frontend Tech Stack

The PRD specifies React, Tailwind CSS and react-i18next. The following adds practical libraries for routing, data fetching, forms, validation and reporting UI.

Layer	Choice	Reason
Framework	React + TypeScript	Component architecture and type safety
Build	Vite	Fast local development and simple deployment
Styling	Tailwind CSS	Explicitly required by the PRD
UI	shadcn/ui	Reusable tables, dialogs, forms and cards
Routing	React Router	Separate internal app and customer/vendor portal routes
Data	TanStack Query	API fetching, caching and invalidation
Forms	React Hook Form + Zod	Fast forms and strong validation
Charts	Recharts	Dashboard, P&L; and budget visuals
Localization	react-i18next	Required English/Hindi/Gujarati support
Testing	Vitest + React Testing Library	Accounting-critical UI tests
Quality	ESLint + Prettier	Consistent maintainable code

Architecture rule

Pages -> reusable components -> hooks/query layer -> API services -> backend. Keep tax calculations, ledger mutation and double-entry enforcement on the backend/system engine.

2. Complete Page / Route Structure

Area	Route	Roles	Depends on
Auth	/login	All	Authentication
Dashboard	/dashboard	Admin, Accountant	Posted data + reports
Contacts	/master/contacts	Admin, Accountant	None
Products	/master/products	Admin, Accountant	None
Chart of Accounts	/master/accounts	Admin	None
Journals	/master/journals	Admin	Accounts
Analytic Accounts	/master/analytics	Admin	None
Budgets	/master/budgets	Admin, Accountant	Analytic Accounts
Sales Orders	/sales/orders	Admin, Accountant	Contacts + Products
Customer Invoices	/sales/invoices	Admin, Accountant	Sales Orders
Customer Payments	/sales/payments	Admin, Accountant	Invoices + Cash/Bank
Purchase Orders	/purchases/orders	Admin, Accountant	Contacts + Products
Vendor Bills	/purchases/bills	Admin, Accountant	Purchase Orders
Vendor Payments	/purchases/payments	Admin, Accountant	Bills + Cash/Bank
Journal Entries	/accounting/journal-entries	Admin	Transactions + Journals + Accounts
Ledgers	/accounting/ledgers	Admin, Accountant	Posted journal entries
Profit & Loss	/reports/profit-loss	Admin, Accountant	Posted ledger data
Balance Sheet	/reports/balance-sheet	Admin, Accountant	Posted ledger data
Budget Report	/reports/budget	Admin, Accountant	Budgets + analytic transactions
Portal	/portal/*	Customer/Vendor	Own documents + payment

Navigation principle: Master Data -> Transactions -> Accounting -> Reports. The portal is a separate restricted route group.

3. Master Data Pages

Master data is the baseline directory that drives operational workflows. These pages should be completed before sales and purchase forms.

Page	Required UI	Key fields	Feeds
Contacts	Table, search, filter, Add/Edit	Name, type Customer/Vendor/Both, email, mobile, address, profile image	Sales, Purchases, Portal
Products	Table, category/type filters, Add/Edit	Name, type Goods/Service/Combo, sales price, cost, category	Sales, Purchases
Chart of Accounts	Account list/tree, Add/Edit	Account name, Asset/Liability/Expense/Income/Capital	Journal Entries, Reports
Journals	Journal list, Add/Edit	Name, type, default accounts; Sales/Purchase/Bank/Cash	Accounting
Analytic Accounts	List, Add/Edit	Name, Income/Expense	Budgets, analytics
Budgets	List, Add/Edit, status	Name, period, responsible person, planned amount, analytic account	Budget Report

Reusable components

DataTable, SearchBar, FilterBar, FormDrawer, SelectWithSearch, ConfirmDialog, StatusBadge, EmptyState and LoadingState.

4. Sales & Purchase Pages

The PRD connects operational documents in sequence. Avoid duplicate data entry when moving from one page to the next.

Sales flow

Page	Requirements	Next
Sales Orders	Customer + product(s) + quantity + unit price + tax; subtotal/tax/total; draft/confirm	Customer Invoice
Customer Invoices	Inherit Sales Order data; invoice date/due date; tax; total; status; register payment	Customer Payment
Customer Payments	Invoice, amount, date, Cash/Bank account; validate amount	Journal Entry / Ledger

Purchase flow

Page	Requirements	Next
Purchase Orders	Vendor + product(s) + quantity + unit price; totals; confirmation	Vendor Bill
Vendor Bills	Inherit PO data; invoice date/due date; total; status; register payment	Vendor Payment
Vendor Payments	Bill, amount, date, Cash/Bank account; validate amount	Journal Entry / Ledger

Key UX rule

A Customer Invoice should inherit the confirmed Sales Order. A Vendor Bill should inherit the Purchase Order. Payments should inherit the open balance from the invoice/bill.

5. Accounting, Ledger & Reporting Pages

Page	Purpose	Frontend requirements
Journal Entries	Actual accounting record	Date, reference, journal, journal items, account, debit, credit, total debit/credit, posting status
Ledgers	Account-wise history	Account selector, date range, debit/credit, running balance, source reference
Profit & Loss	Income minus purchases/expenses = net profit	Period filter, account/category grouping, export/print UI
Balance Sheet	Assets, liabilities and capital/equity	As-of date, expandable account groups, export/print UI
Budget Report	Planned vs actual with variance	Period, analytic account, responsible person, variance view

Double-entry rule

Every transaction must satisfy $\text{Total Debits} = \text{Total Credits}$. The frontend can show the totals and validation state, but the backend/system engine must enforce the accounting rule.

Typical sale

Event	Debit	Credit
Customer invoice	Customer / Debtor	Sales Income
Customer payment	Cash or Bank	Customer / Debtor

Typical purchase

Event	Debit	Credit
Vendor bill	Purchase Expense	Creditor
Vendor payment	Creditor	Cash or Bank

6. Dashboard + Customer/Vendor Portal

The PRD explicitly requires the three financial reports. A dashboard is recommended as a summary layer over the same authoritative data.

Page	Content	Main interactions
Dashboard	Sales, purchases, receivables, payables, net profit, payment status, recent transactions; optional charts	Date range, quick actions, status filters
Portal Home	Summary of the logged-in contact's own documents	My Invoices / My Bills
My Invoices	Own customer invoices only	View detail -> payment
My Bills	Own vendor bills where applicable	View detail -> payment/status
Transaction Detail	Document details and status	Back / Pay
Payment	Initiate eligible payment	Confirmation -> updated status

Role-aware navigation

Capability	Admin	Accountant	Customer/Vendor
Manage master data	Yes	Yes	No
Record sales/purchases	Yes	Yes	No
Register payments	Yes	Yes	Own payment initiation
Accounting/ledger	Yes	As permitted	No
Financial reports	Yes	Standard reports	No
Own invoices/bills	All	All	Own only

Security rule

Frontend role checks improve UX but are not the security boundary. Backend authorization and record ownership must be enforced.

7. End-to-End Build & Demo Flow

Stage	Pages	Output
1	Login	Authenticated user + role
2	Contacts, Products, Accounts, Journals, Analytics, Budgets	Master data ready for selection
3	Sales Order -> Customer Invoice -> Customer Payment	Sales transaction + payment
4	Purchase Order -> Vendor Bill -> Vendor Payment	Purchase transaction + payment
5	Journal Entries -> Ledgers	Balanced double-entry + account balances
6	P&L; -> Balance Sheet -> Budget Report	Financial statements and variance
7	Customer/Vendor Portal	Own documents, status and payment

The complete chain

MASTER DATA -> SALES / PURCHASES -> INVOICE / BILL -> PAYMENT -> JOURNAL ENTRY -> LEDGER -> REPORTS

Recommended implementation order

1. App shell + authentication/roles 2. Master Data 3. Sales Order + Invoice + Payment 4. Purchase Order + Bill + Payment 5. Journal Entries + Ledger views 6. P&L; + Balance Sheet + Budget Report 7. Customer/Vendor portal 8. Localization, testing and final UI polish

Final reviewer story

Create a Contact and Product -> create a Sales Order -> generate the Customer Invoice -> record payment -> open the Journal Entry/Ledger -> open P&L; or Balance Sheet. Repeat for a purchase, then show Budget Report and the restricted portal.

Source basis: Urban_Furniture_Accounting_System_PRD.pdf. The PRD defines the system objective, roles, master data, transaction flows, analytics/budgets, reports and React + Tailwind CSS + react-i18next direction. Additional libraries in this document are implementation recommendations.